Ex. No: 1	SOFTWARE REQUIREMENTS SPECIFICATION (SRS)
Date:	

Software Requirements Specification (SRS)

Employee Payroll Management System

1. Introduction

1.1 Purpose

The Employee Payroll Management System is a web-based application that allows organizations to manage employee information, attendance, salary details, deductions, allowances, and payroll processing. The system reduces manual payroll calculations and provides a simple and efficient way to calculate and manage employee salaries. The system also provides an administrator module through which the administrator can manage employees, attendance, salary structures, deductions, and payroll records.

1.2 Scope

The system provides the following major functionalities:

- Employee registration and management
- User login and authentication
- Employee profile management
- Attendance management
- Salary structure management
- Allowance and deduction management
- Automatic salary calculation
- Payroll generation
- Payslip generation
- Viewing payroll history
- Administrator management of employees and payroll

The system can be adapted for different organizations such as companies, educational institutions, hospitals, industries, and other organizations that require automated employee salary management.

1.3 Intended Users

The system has two main types of users:

Employee:

A registered employee can view their profile, attendance, salary details, generated payslips, and payroll history.

Administrator:

The administrator can add, update, and delete employee information, manage attendance, define salary structures, manage allowances and deductions, process payroll, and generate payroll reports.

2. Overall Description

2.1 Product Perspective

The Employee Payroll Management System is a standalone web application consisting of a frontend, backend, and database. The frontend provides the user interface for registration, login, employee management, attendance, and payroll operations. The backend handles business logic, authentication, salary calculations, payroll processing, and report generation. The database stores employee, attendance, salary, deduction, allowance, payroll, and payslip information.

2.2 Product Functions

The major functions of the system are:

1. Registration and Login

o Users can create an employee account.

o Registered users can log in using their credentials.

o The system authenticates users before allowing access to employee or payroll information.

2. Employee Management

o Administrators can add new employees.

o Administrators can edit or remove employee information.

o Employee details such as employee ID, name, department, designation, contact information, and joining date can be maintained.

3. Attendance Management

o Administrators can record employee attendance.

o The system can maintain working days, present days, absent days, and leave details.

o Attendance information can be used during payroll calculation.

4. Salary Management

o Administrators can define employee salary structures.

o The system can maintain basic salary, allowances, bonuses, and deductions.

o Salary components can be updated when required.

5. Payroll Processing

o The system shall calculate employee salaries based on salary components, attendance, allowances, and deductions.

o The system shall calculate the net salary automatically.

o Payroll records shall be stored in the database.

6. Payslip Generation

o The system can generate a payslip for each employee.

o The payslip can contain employee details, salary components, deductions, and net salary.

o Employees can view or download their payslips.

7. Payroll Management

o Employees can view their current and previous payroll records.

o Administrators can view and manage payroll records for all employees.

o The system shall maintain payroll history for future reference.

8. Notifications

o The system can display a confirmation message after successful payroll processing.

o Employees can be notified when their payslip is generated.

o Payroll and salary-related information can be displayed to the employee.

3. Functional Requirements

FR1 – Employee Registration

The system shall allow new employees to register by providing required information such as employee ID, name, email, phone number, department, and password.

FR2 – User Login

The system shall authenticate users using their registered email/username and password.

FR3 – Employee Management

The system shall allow administrators to add, update, view, and delete employee information.

FR4 – View Employee Details

The system shall allow authorized users to view employee information such as employee ID, name, department, designation, and contact details.

FR5 – Attendance Management

The system shall allow administrators to record and manage employee attendance and leave information.

FR6 – Salary Structure

The system shall allow administrators to define and maintain employee salary components such as basic salary, allowances, bonuses, and deductions.

FR7 – Salary Calculation

The system shall automatically calculate the employee's gross salary and net salary based on the applicable salary components.

FR8 – Payroll Processing

The system shall allow administrators to process monthly payroll for employees.

FR9 – Payslip Generation

The system shall generate a payslip containing employee details, salary components, deductions, and net salary.

FR10 – View Payroll

Employees shall be able to view their current and previous payroll details.

FR11 – Payroll History

The system shall maintain previous payroll records for each employee.

FR12 – Admin Management

The administrator shall be able to manage employees, attendance, salary structures, deductions, allowances, payroll, and payslips.

\

4. Non-Functional Requirements

4.1 Performance

• The system should respond to user requests quickly.

• Payroll calculations should be completed within a reasonable time.

• The system should support multiple employees being processed simultaneously.

4.2 Security

• User passwords shall be securely stored.

• Only authenticated users shall be allowed to access payroll information.

• Employees shall only be allowed to view their own salary and payroll details.

• Administrator functions shall be accessible only to authorized administrators.

4.3 Usability

• The interface should be simple and easy to understand.

• Administrators should be able to process payroll with minimum steps.

• Employees should be able to easily view their salary and payslip information.

• Appropriate error messages should be displayed for invalid inputs.

4.4 Reliability

• The system should prevent incorrect or duplicate payroll records.

• Employee and payroll information should be stored accurately.

• The system should handle invalid requests without crashing.

• Salary calculations should be performed consistently based on the defined salary structure.

4.5 Maintainability

• The application should use modular components.

• The database and application code should be easy to update.

• New salary components and payroll features should be added without major changes to the existing system.

5. External Interface Requirements

5.1 User Interface

The system shall provide the following pages:

- Home Page
- Employee Registration Page
- Login Page
- Employee Dashboard
- Employee Profile Page
- Attendance Page
- Salary Details Page
- Payroll Processing Page
- Payslip Page

- Payroll History Page
- Admin Dashboard
- Employee Management Page

5.2 Software Interface

The application shall communicate between the frontend and backend using APIs. The backend shall communicate with the database to store and retrieve employee, attendance, salary, and payroll information.

The system shall generate payroll and payslip information based on the employee records stored in the database.

6. System Constraints

- The user must have an account to access employee-specific payroll information.
- Only authorized administrators can process payroll.
- Required employee and salary fields must be completed before payroll processing.
- Salary calculations depend on the employee's defined salary structure and applicable deductions and allowances.
- An employee cannot access another employee's payroll information.
- The system requires a database connection for storing employee and payroll information.

7. Assumptions

- Employees provide valid personal and employment information during registration.
- The administrator maintains correct employee, attendance, salary, allowance, and deduction details.
- The organization defines the applicable salary structure and payroll rules.
- Users have access to a web browser and internet/network connection.
- Each employee is associated with a unique employee ID.
- Each payroll record is associated with one employee and a specific payroll period.

8. Expected Outcome

The Employee Payroll Management System will provide a simple and efficient platform for managing employee payroll. It will reduce manual salary calculations, minimize payroll errors, maintain accurate employee and salary records, automate payroll processing, generate payslips, and allow employees and administrators to access payroll information through a centralized system.

EXP-1: AUTOMATING FILE MANAGEMENT (PYTHON)

DATE:

AIM:

To automate the organisation of files in a folder by moving them into category-wise sub-folders based on their file extension, renaming them with a date prefix and logging every operation using Python.

ALGORITHM:

1. Start the program.
2. Import the os, shutil, logging and datetime modules.
3. Define the file categories and their extensions (Documents, Images, Spreadsheets, Code, Archives).
4. Configure logging to write into file_manager.log.
5. Create sample files in the source folder.
6. Read every file in the source folder using os.listdir().
7. Find the extension of each file and identify its category; unknown extensions go to 'Others'.
8. Create the category folder if it does not exist.
9. Rename the file with the current date prefix and move it using shutil.move().
10. Log the operation and update the summary count.
11. Display the summary of files organised.
12. Stop the program.

CODE:

FILE_MANAGER.PY

```
import os
import shutil
import logging
from datetime import datetime

SOURCE = "Downloads_Demo"
CATEGORIES = {
    "Documents": [".pdf", ".docx", ".txt", ".pptx"],
    "Images": [".jpg", ".jpeg", ".png", ".gif"],
    "Spreadsheets": [".xlsx", ".csv"],
    "Code": [".py", ".java", ".c", ".html"],
    "Archives": [".zip", ".rar"],
}

logging.basicConfig(filename="file_manager.log", level=logging.INFO,
                    format="%(asctime)s - %(levelname)s - %(message)s")


def create_sample_files():
    samples = ["HCI_Notes.pdf", "Resume.docx", "photo1.jpg", "screenshot.png",
               "marks.xlsx", "attendance.csv", "main.py", "Library.java",
               "project_backup.zip", "todo.txt", "unknown.xyz"]
    os.makedirs(SOURCE, exist_ok=True)
```

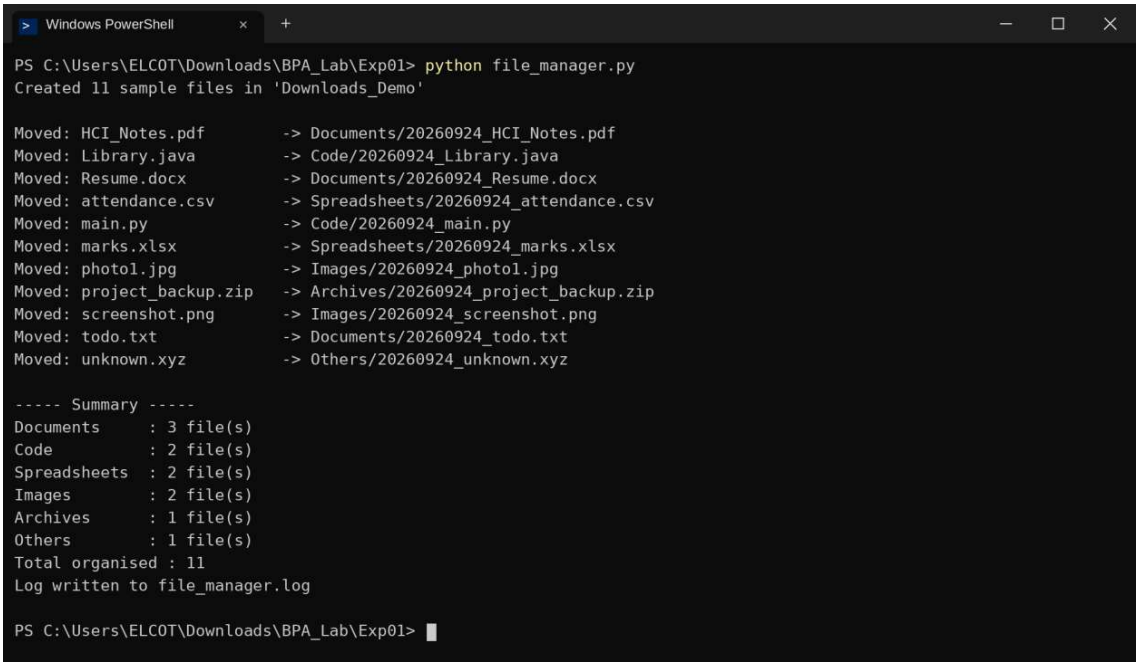
```
    for name in samples:
        open(os.path.join(SOURCE, name), "w").close()
    print(f"Created {len(samples)} sample files in '{SOURCE}'\n")


def get_category(ext):
    for category, extensions in CATEGORIES.items():
        if ext.lower() in extensions:
            return category
    return "Others"

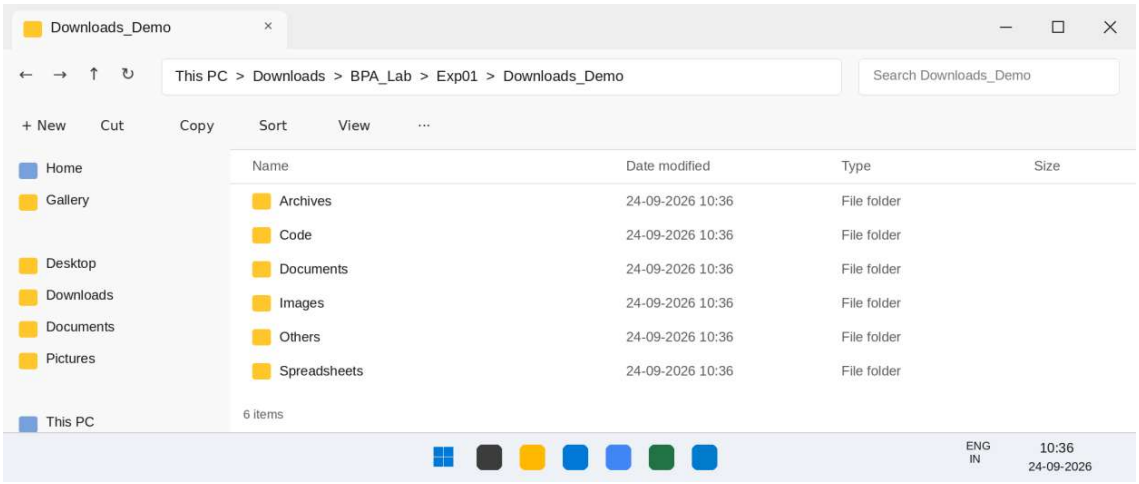

def organize(folder):
    summary = {}
    stamp = datetime.now().strftime("%Y%m%d")
    for name in sorted(os.listdir(folder)):
        path = os.path.join(folder, name)
        if not os.path.isfile(path):
            continue
        category = get_category(os.path.splitext(name)[1])
        os.makedirs(os.path.join(folder, category), exist_ok=True)
        new_name = f"{stamp}_{name}"
        shutil.move(path, os.path.join(folder, category, new_name))
        logging.info(f"Moved {name} -> {category}/{new_name}")
        print(f"Moved: {name:<20} -> {category}/{new_name}")
        summary[category] = summary.get(category, 0) + 1
    return summary


if __name__ == "__main__":
    create_sample_files()
    result = organize(SOURCE)
    print("\n----- Summary -----")
    for category, count in result.items():
        print(f"{category:<14}: {count} file(s)")
    print(f"Total organised : {sum(result.values())}")
    print("Log written to file_manager.log")
```

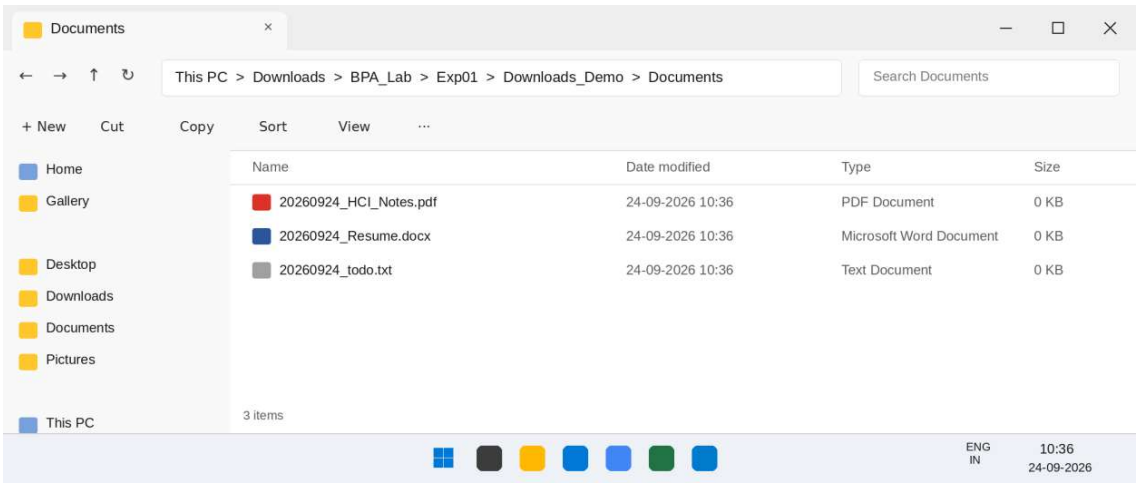
OUTPUT:



Program execution in PowerShell



Category folders created automatically



Renamed files inside the Documents folder

RESULT:

Thus, the file management process was successfully automated using Python. The files were automatically classified by extension, renamed with a date prefix, moved into category folders, and every operation was recorded in a log file.

EXP-2: WEB SCRAPING AND CSV EXPORT (PYTHON)

DATE:

AIM:

To scrape book details such as title, price, rating and stock availability from a website using Requests and BeautifulSoup and export the collected data into a CSV file.

ALGORITHM:

1. Start the program.
2. Import the requests, BeautifulSoup and csv modules.
3. Specify the URL of the web pages to be scraped.
4. Send an HTTP GET request to each page using requests.get().
5. Parse the HTML content using BeautifulSoup.
6. Locate every book item using the tag <article class='product_pod'>.
7. Extract the title, price, star rating and stock status of each book.
8. Store the details of all pages in a list of dictionaries.
9. Write the list into books.csv using csv.DictWriter.
10. Display the number of records and the first five records.
11. Stop the program.

CODE:

SCRAPE_CSV.PY

```
import csv
import requests
from bs4 import BeautifulSoup

URL = "https://books.toscrape.com/catalogue/page-{}.html"
RATINGS = {"One": 1, "Two": 2, "Three": 3, "Four": 4, "Five": 5}


def scrape_page(page):
    response = requests.get(URL.format(page), timeout=10)
    response.encoding = "utf-8"
    soup = BeautifulSoup(response.text, "html.parser")
    books = []
    for item in soup.find_all("article", class_="product_pod"):
        books.append({
            "Title": item.h3.a["title"],
            "Price": float(item.find("p", class_="price_color").text[1:]),
            "Rating": RATINGS[item.find("p", class_="star-rating")["class"][1]],
            "Stock": item.find("p", class_="instock availability").text.strip(),
        })
    return books


if __name__ == "__main__":
    all_books = []
    for page in range(1, 4):
        books = scrape_page(page)
        print(f"Page {page} scraped - {len(books)} books found")
        all_books.extend(books)
```

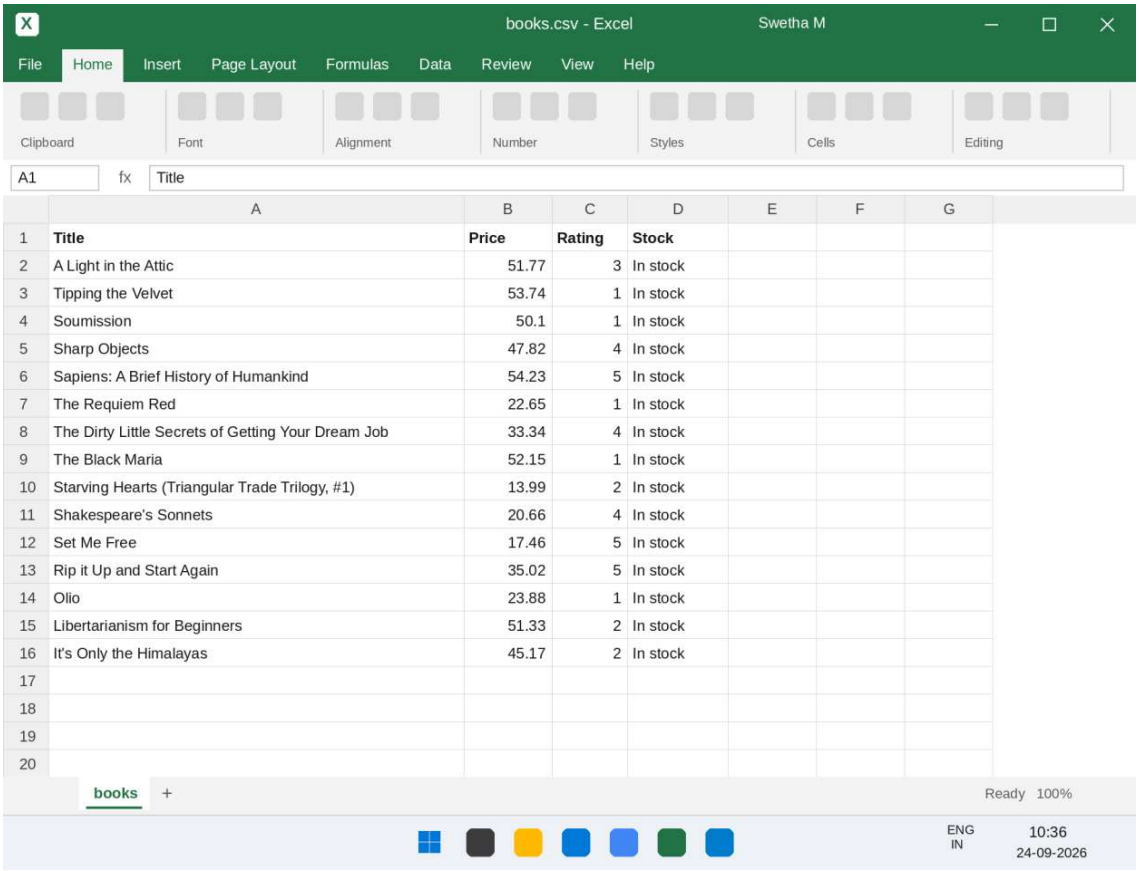
```
    with open("books.csv", "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["Title", "Price", "Rating", "Stock"])
        writer.writeheader()
        writer.writerows(all_books)

    print(f"\nTotal {len(all_books)} records exported to books.csv")
    print("\nFirst 5 records:")
    for b in all_books[:5]:
        print(f"  {b['Title'][:38]:<38}  GBP {b['Price']:>6}  {b['Rating']} star")
```

OUTPUT:



Program execution in PowerShell



Exported books.csv opened in Excel

RESULT:

Thus, the book details were successfully scraped from the website using Requests and BeautifulSoup and exported into a CSV file.

EXP-3: AUTOMATED EMAIL SENDING (PYTHON)

DATE:

AIM:

To automatically send personalised reminder emails to a list of recipients stored in a CSV file using Python's smtplib module.

ALGORITHM:

1. Start the program.
2. Import the os, csv, smtplib and EmailMessage modules.
3. Read the sender email and Gmail App Password from environment variables.
4. Prepare the email body template with placeholders for name, subject and due date.
5. Connect to smtp.gmail.com on port 587 and start TLS encryption.
6. Log in to the mail server using the sender credentials.
7. Read the recipient details from recipients.csv.
8. For each recipient, create an EmailMessage with the personalised subject and body.
9. Send the message using send_message() and display the status.
10. Close the SMTP connection.
11. Stop the program.

CODE:

EMAIL_SENDER.PY

```
import os
import csv
import smtplib
from email.message import EmailMessage

SENDER = os.getenv("EMAIL_USER")        # your Gmail address
PASSWORD = os.getenv("EMAIL_PASS")      # 16-character Gmail App Password

TEMPLATE = """Dear {name},

This is a reminder that your {subject} record must be submitted on or
before {due_date}. Please bring the printed record to the lab.

Regards,
BPA Lab Automation Bot
"""


def send_mails(csv_file):
    with smtplib.SMTP("smtp.gmail.com", 587) as server:
        server.starttls()
        server.login(SENDER, PASSWORD)
        print("Connected and logged in to smtp.gmail.com\n")

        with open(csv_file) as f:
            rows = list(csv.DictReader(f))
        for i, row in enumerate(rows, start=1):
            msg = EmailMessage()
            msg["From"] = SENDER
            msg["To"] = row["email"]
```

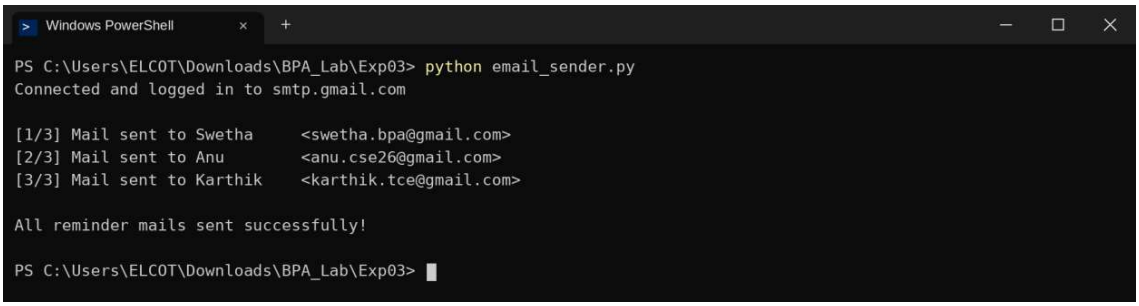
```
            msg["Subject"] = f"Reminder: {row['subject']} Record Submission"
            msg.set_content(TEMPLATE.format(**row))
            server.send_message(msg)
            print(f"[{i}/{len(rows)}] Mail sent to {row['name']:<10}
<{row['email']}>")
    print("\nAll reminder mails sent successfully!")


if __name__ == "__main__":
    send_mails("recipients.csv")
```

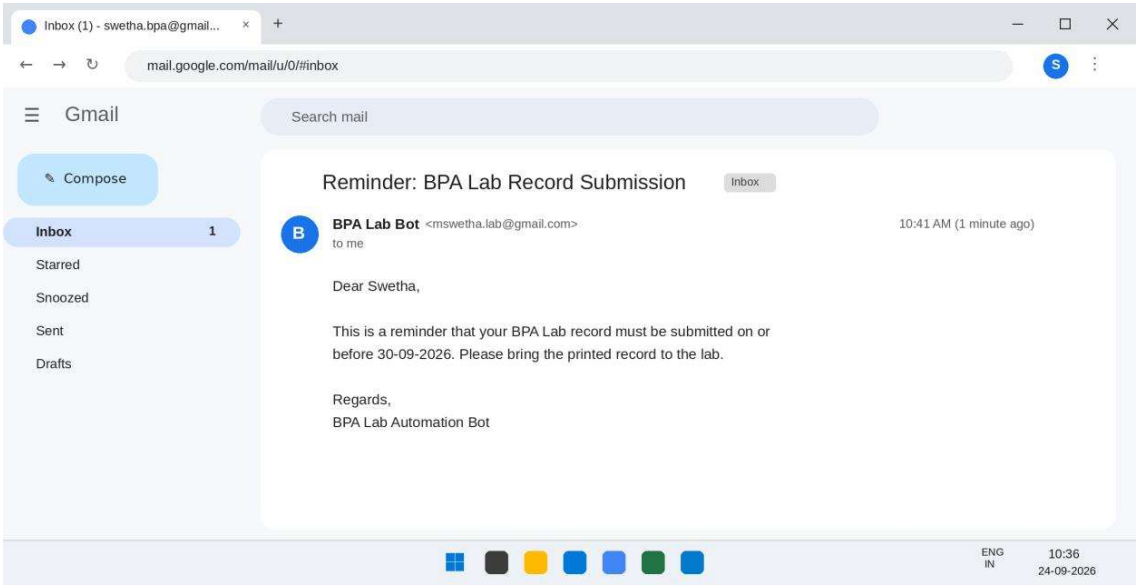
RECIPIENTS.CSV (INPUT)

```
name,email,subject,due_date
Swetha,swetha.bpa@gmail.com,BPA Lab,30-09-2026
Anu,anu.cse26@gmail.com,BPA Lab,30-09-2026
Karthik,karthik.tce@gmail.com,BPA Lab,30-09-2026
```

OUTPUT:



Program execution in PowerShell



Reminder mail received in the recipient's Gmail inbox

RESULT:

Thus, personalised reminder emails were successfully sent to all recipients in the CSV file automatically using Python and SMTP.

EXP-4: READING PDF AND EXTRACTING DATA (PYTHON)

DATE:

AIM:

To read a PDF document and extract its text, specific fields such as name, register number, email, phone and date, and tabular data using Python.

ALGORITHM:

1. Start the program.
2. Import the pdfplumber, re and csv modules.
3. Open the PDF file using pdfplumber.open().
4. Display the total number of pages.
5. Extract text from every page using extract_text().
6. Extract tables from every page using extract_tables().
7. Display the complete extracted text.
8. Define regular expression patterns for the required fields.
9. Search the text using re.search() and display each field.
10. Save the extracted table into courses.csv.
11. Stop the program.

CODE:

PDF_READER.PY

```
import re
import csv
import pdfplumber

PDF_FILE = "student_form.pdf"

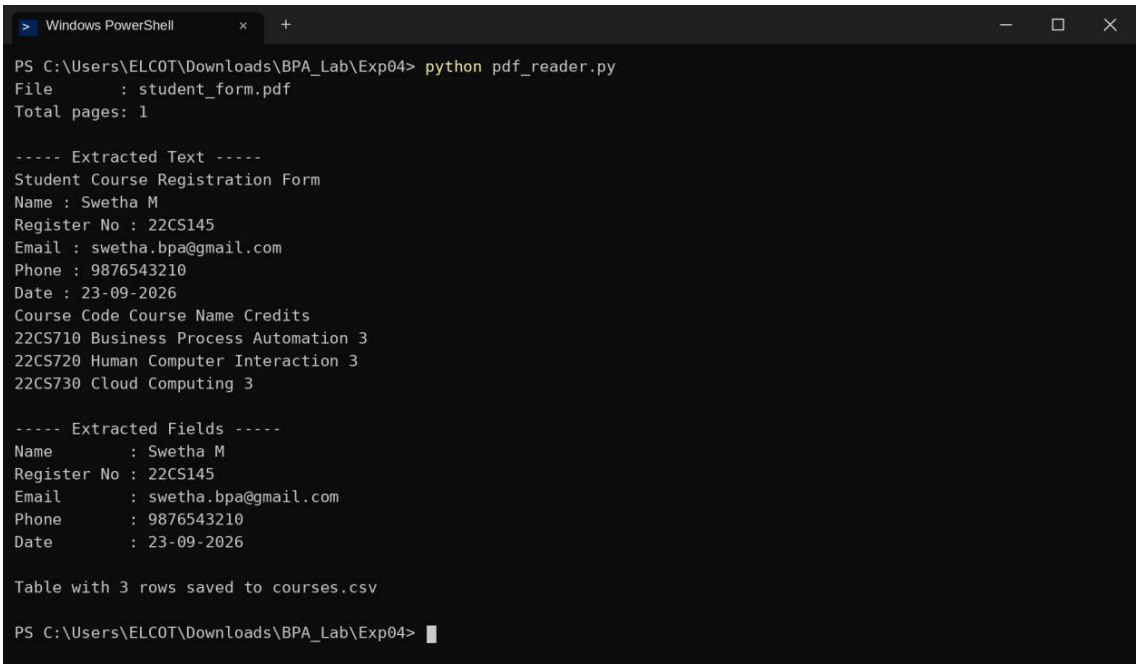
with pdfplumber.open(PDF_FILE) as pdf:
    print(f"File       : {PDF_FILE}")
    print(f"Total pages: {len(pdf.pages)}\n")
    text = ""
    tables = []
    for page in pdf.pages:
        text += page.extract_text() + "\n"
        tables.extend(page.extract_tables())

print("----- Extracted Text -----")
print(text)

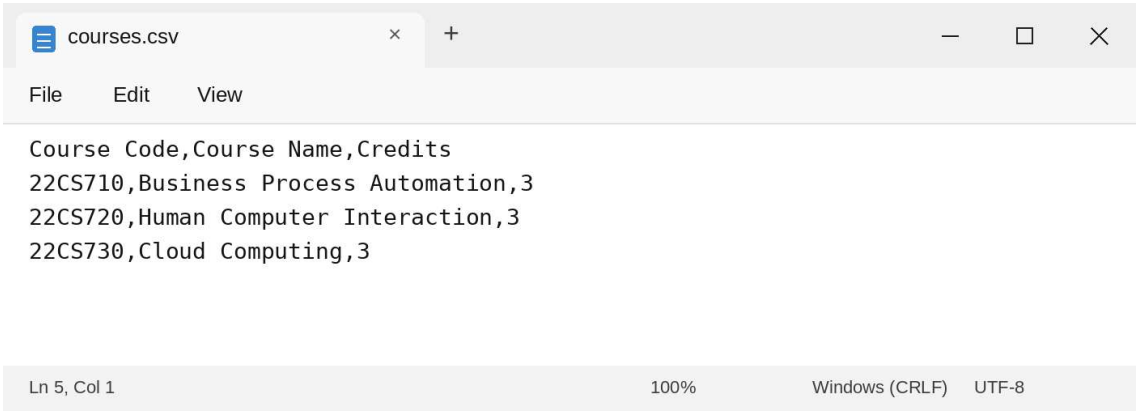
patterns = {
    "Name": r"Name\s*:\s*(.+)",
    "Register No": r"Register No\s*:\s*(\S+)",
    "Email": r"([\w.]+@[\w.]+\.\w+)",
    "Phone": r"(\d{10})",
    "Date": r"(\d{2}-\d{2}-\d{4})",
}
print("----- Extracted Fields -----")
for field, pattern in patterns.items():
    match = re.search(pattern, text)
    print(f"{field:<12}: {match.group(1).strip() if match else 'Not found'}")
```

```
with open("courses.csv", "w", newline="") as f:
    csv.writer(f).writerows(tables[0])
print(f"\nTable with {len(tables[0]) - 1} rows saved to courses.csv")
```

OUTPUT:



Program execution in PowerShell



Extracted table saved as courses.csv

RESULT:

Thus, the text, required fields and table were successfully extracted from the PDF document using pdfplumber and regular expressions.

EXP-5: DESKTOP AUTOMATION USING PYAUTOGUI (PYTHON)

DATE:

AIM:

To automate a desktop task using PyAutoGUI by opening Notepad, typing a lab attendance report, saving it as a file and capturing a screenshot automatically.

ALGORITHM:

1. Start the program.
2. Import the pyautogui and time modules.
3. Enable the FAILSAFE feature and set a pause between actions.
4. Prepare the attendance report as a list of lines.
5. Open the Run dialog using Win + R and launch Notepad.
6. Type each line of the report using pyautogui.write() and press Enter.
7. Press Ctrl + S to open the Save dialog.
8. Type the file name attendance_report.txt and press Enter.
9. Capture a screenshot using pyautogui.screenshot().
10. Display the completion message.
11. Stop the program.

CODE:

DESKTOP_BOT.PY

```
import time
import pyautogui

pyautogui.FAILSAFE = True      # move mouse to top-left corner to stop
pyautogui.PAUSE = 0.3

report = [
    "LAB ATTENDANCE SUMMARY",
    "======================",
    "Course      : Business Process Automation",
    "Class       : VII CSE - A",
    "Total       : 62",
    "Present     : 57",
    "Absent      : 5",
    "Attendance  : 91.94 %",
    "Generated by: PyAutoGUI Desktop Bot",
]

print("Bot starts in 3 seconds. Do not use mouse/keyboard...")
time.sleep(3)

# 1. Open Notepad through the Run dialog
pyautogui.hotkey("win", "r")
time.sleep(1)
pyautogui.write("notepad", interval=0.05)
pyautogui.press("enter")
time.sleep(2)

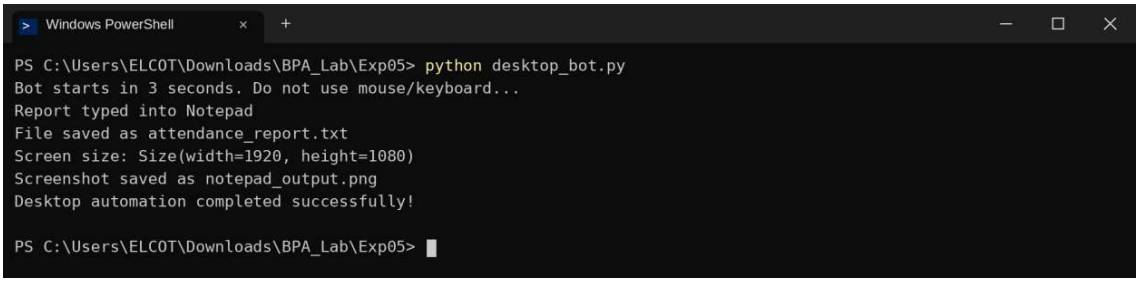
# 2. Type the report line by line
for line in report:
```

```
    pyautogui.write(line, interval=0.02)
    pyautogui.press("enter")
print("Report typed into Notepad")

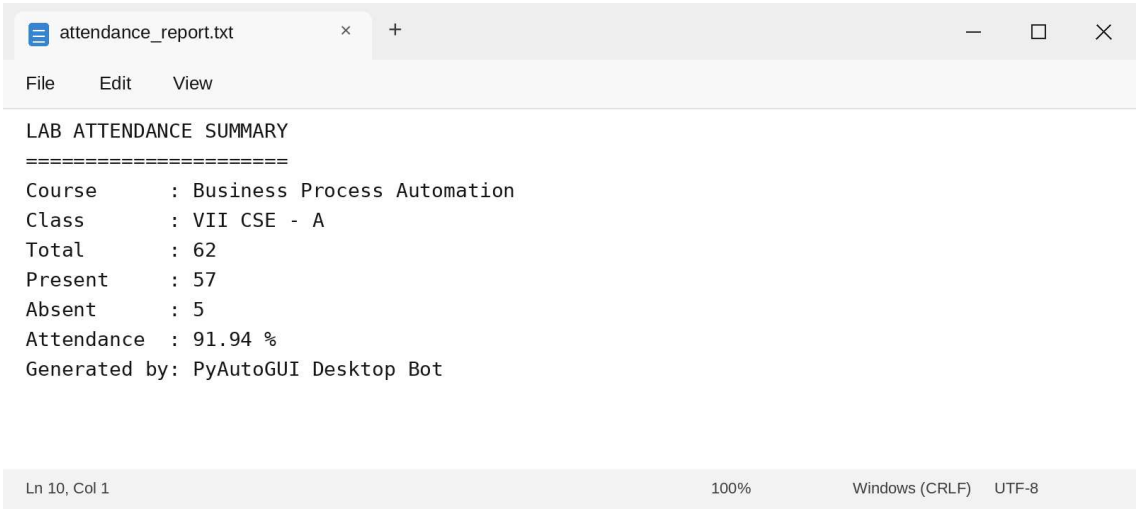
# 3. Save the file
pyautogui.hotkey("ctrl", "s")
time.sleep(1.5)
pyautogui.write("attendance_report.txt", interval=0.03)
pyautogui.press("enter")
time.sleep(1)
print("File saved as attendance_report.txt")

# 4. Screenshot of the result
pyautogui.screenshot("notepad_output.png")
print(f"Screen size: {pyautogui.size()}")
print("Screenshot saved as notepad_output.png")
print("Desktop automation completed successfully!")
```

OUTPUT:



Program execution in PowerShell



Report typed and saved in Notepad automatically

RESULT:

Thus, the desktop task was successfully automated using PyAutoGUI. Notepad was opened, the attendance report was typed and saved as a file, and a screenshot was captured without manual intervention.

EXP-6: WEB FORM AUTOMATION WITH SELENIUM (PYTHON)

DATE:

AIM:

To automate filling and submitting a web form for multiple records read from a CSV file using Selenium WebDriver.

ALGORITHM:

1. Start the program.
2. Import webdriver, By, Select and the csv module.
3. Launch the Chrome browser using webdriver.Chrome() and maximise it.
4. Read the form data from form_data.csv.
5. For each record, open the web form URL.
6. Locate the text, password and textarea fields and enter the values.
7. Select the dropdown option and enter the date.
8. Click the checkbox and radio button.
9. Click the Submit button.
10. Read the confirmation message, save a screenshot and display the status.
11. Close the browser after all records are processed.
12. Stop the program.

CODE:

FORM_BOT.PY

```
import csv
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select

URL = "https://www.selenium.dev/selenium/web/web-form.html"

driver = webdriver.Chrome()
driver.maximize_window()

with open("form_data.csv") as f:
    for i, row in enumerate(csv.DictReader(f), start=1):
        driver.get(URL)
        driver.find_element(By.NAME, "my-text").send_keys(row["name"])
        driver.find_element(By.NAME, "my-password").send_keys(row["password"])
        driver.find_element(By.NAME, "my-textarea").send_keys(row["comments"])
        Select(driver.find_element(By.NAME, "my-
select")).select_by_visible_text(row["option"])
        driver.find_element(By.NAME, "my-date").send_keys(row["date"])
        driver.find_element(By.ID, "my-check-2").click()
        driver.find_element(By.ID, "my-radio-2").click()
        time.sleep(1)
        driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

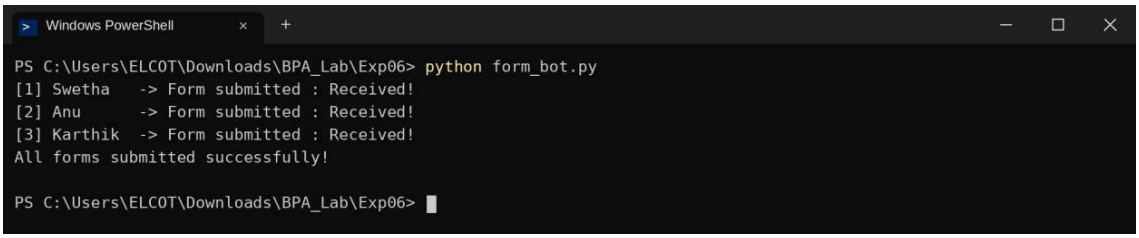
        heading = driver.find_element(By.TAG_NAME, "h1").text
        message = driver.find_element(By.ID, "message").text
        driver.save_screenshot(f"submitted_{i}.png")
        print(f"[{i}] {row['name']:<8} -> {heading} : {message}")
```

```
driver.quit()
print("All forms submitted successfully!")
```

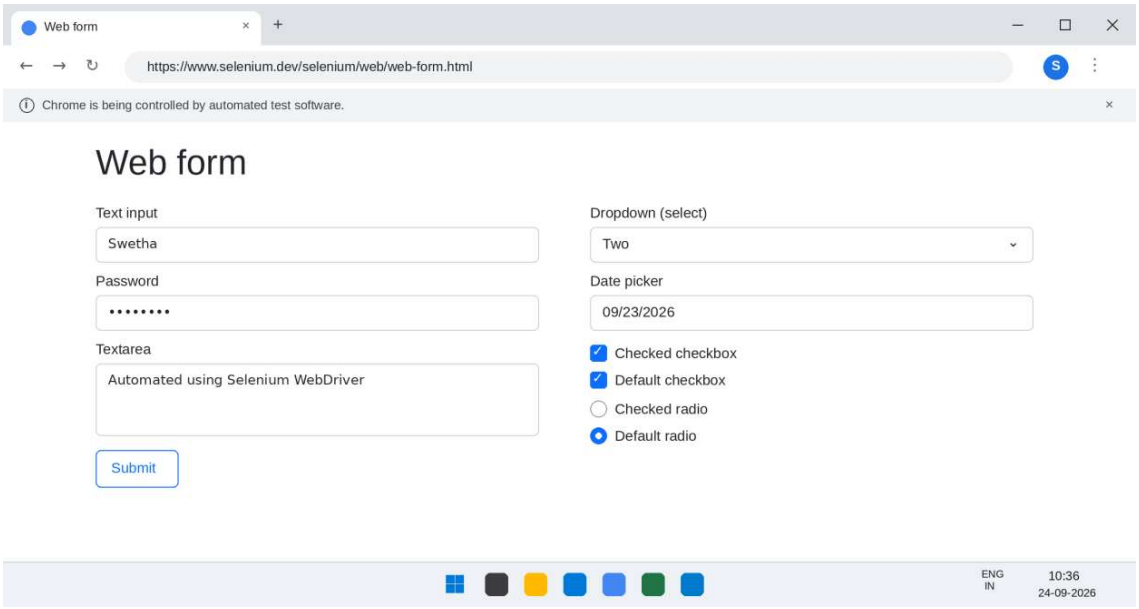
FORM_DATA.CSV (INPUT)

```
name,password,comments,option,date
Swetha,Bpa@2026,Automated using Selenium WebDriver,Two,09/23/2026
Anu,Anu#123,Data driven form filling,One,09/23/2026
Karthik,Kar$456,Web form automation lab,Three,09/23/2026
```

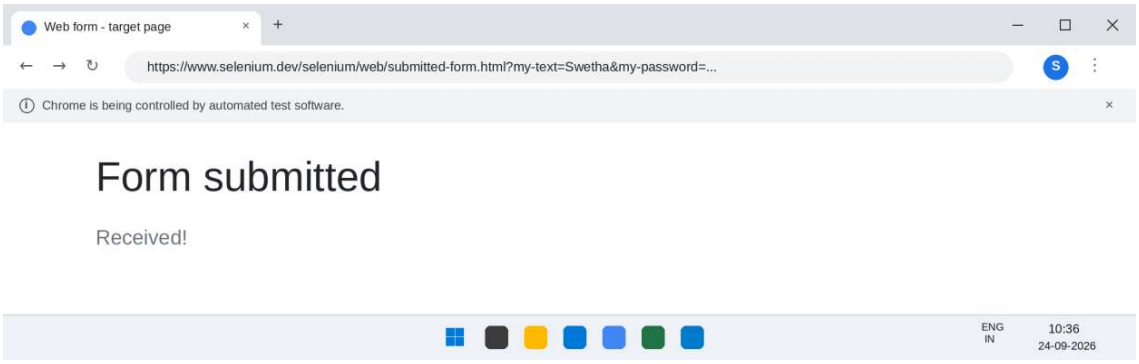
OUTPUT:



Program execution in PowerShell



Form filled automatically by Selenium



Confirmation page after submission

RESULT:

Thus, the web form was filled and submitted automatically for all records in the CSV file using Selenium WebDriver.

EXP-7: HELLO WORLD BOT & VARIABLE HANDLING

DATE:

AIM:

To create a simple Hello World bot that greets the user and to demonstrate variable handling including data types, type conversion, collections and string operations.

ALGORITHM:

1. Start the program.
2. Import the datetime module and declare a global bot name.
3. Display 'Hello World!' and read the user's name.
4. Greet the user based on the current time of day.
5. Declare variables of type String, Integer, Float, Boolean and DateTime and display their types.
6. Perform type conversion using str(), int() and float().
7. Create a list and a dictionary and perform operations on them.
8. Perform string operations such as trim, upper, title, replace, split and contains.
9. Perform arithmetic operations and display the results.
10. Display the goodbye message.
11. Stop the program.

CODE:

HELLO_WORLD_BOT.PY

```
"""
Exp 7: Hello World Bot & Variable Handling
A simple bot that greets the user and demonstrates variable types,
type conversion, collections, string operations and scope.
"""
from datetime import datetime

BOT_NAME = "HelloBot"  # global variable (constant)


def greet(name):
    hour = datetime.now().hour
    greeting = "Good Morning" if hour < 12 else "Good Afternoon" if hour < 17 else
"Good Evening"
    return f"{greeting}, {name}! I am {BOT_NAME}."


def variable_demo():
    # Basic data types
    name = "Swetha"            # String
    age = 21                   # Int32 / Integer
    cgpa = 8.75                # Double / Float
    is_student = True          # Boolean
    today = datetime.now()     # DateTime

    print("\n=== Basic Variables ===")
    for var, val in [("name", name), ("age", age), ("cgpa", cgpa),
                     ("is_student", is_student), ("today", today)]:
```

```
        print(f"{var:<12} = {str(val):<28} type: {type(val).__name__}")

    # Type conversion
    print("\n=== Type Conversion ===")
    age_str = str(age)
    marks = int("95")
    percent = float("88.5")
    print(f"str(age)        -> '{age_str}' ({type(age_str).__name__})")
    print(f"int('95')       -> {marks} ({type(marks).__name__})")
    print(f"float('88.5')   -> {percent} ({type(percent).__name__})")

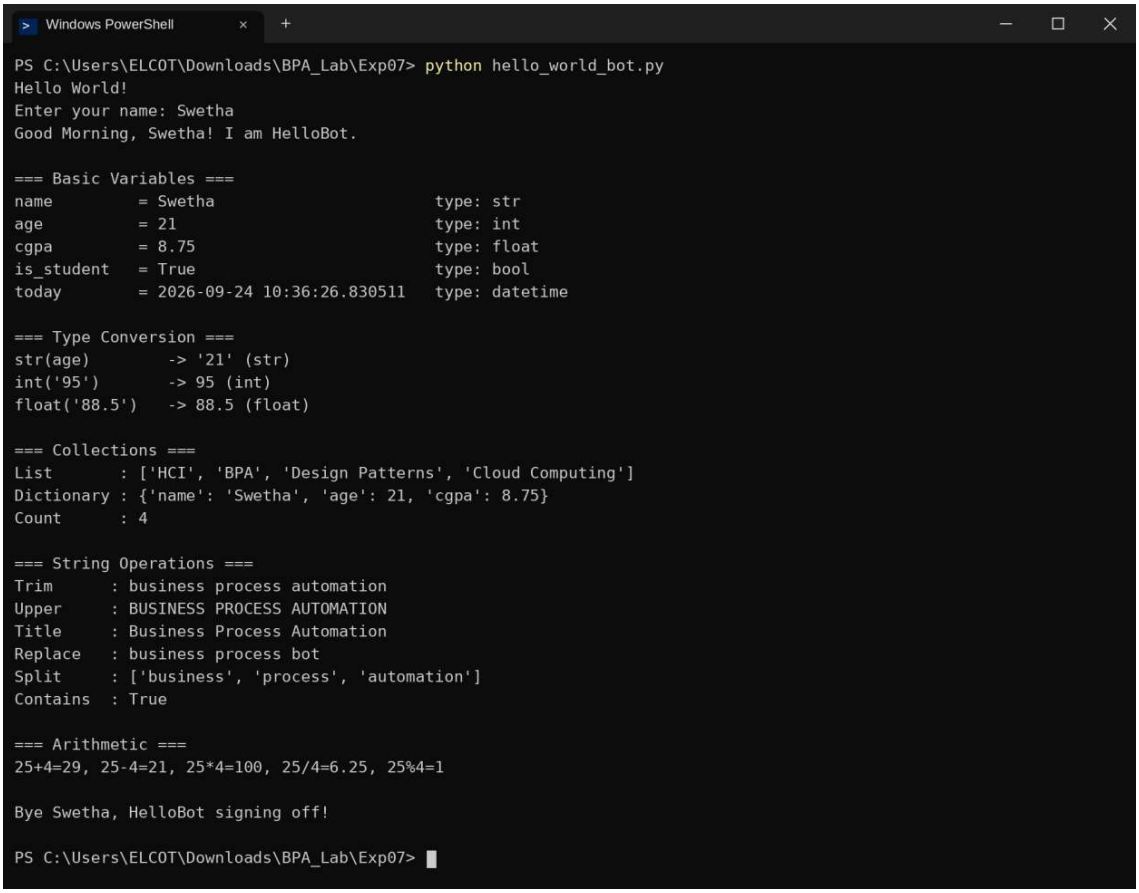
    # Collections
    print("\n=== Collections ===")
    subjects = ["HCI", "BPA", "Design Patterns"]          # Array / List
    student = {"name": name, "age": age, "cgpa": cgpa}    # Dictionary
    subjects.append("Cloud Computing")
    print("List       :", subjects)
    print("Dictionary :", student)
    print("Count      :", len(subjects))

    # String operations
    print("\n=== String Operations ===")
    msg = "  business process automation  "
    print("Trim      :", msg.strip())
    print("Upper     :", msg.strip().upper())
    print("Title     :", msg.strip().title())
    print("Replace   :", msg.strip().replace("automation", "bot"))
    print("Split     :", msg.split())
    print("Contains  :", "process" in msg)

    # Arithmetic
    print("\n=== Arithmetic ===")
    a, b = 25, 4
    print(f"{a}+{b}={a+b}, {a}-{b}={a-b}, {a}*{b}={a*b}, {a}/{b}={a/b},
{a}%{b}={a%b}")


if __name__ == "__main__":
    print("Hello World!")
    user = input("Enter your name: ").strip() or "User"
    print(greet(user))
    variable_demo()
    print(f"\nBye {user}, {BOT_NAME} signing off!")
```

OUTPUT:



Program execution in PowerShell

RESULT:

Thus, the Hello World bot was created successfully and different types of variables, type conversions, collections and string operations were demonstrated.

EXP-8: WEB DATA SCRAPING TO EXCEL

DATE:

AIM:

To scrape tabular data of Indian states by population from a Wikipedia web page, clean it and export it into a formatted Excel file using Python.

ALGORITHM:

1. Start the program.
2. Import requests, pandas and openpyxl.
3. Download the web page using requests with a User-Agent header.
4. Read all HTML tables from the page using pandas.read_html().
5. Select the required table and the State and Population columns.
6. Clean the population values by removing commas and reference marks.
7. Sort by population, take the top 15 states and add Rank and Share (%) columns.
8. Export the DataFrame to india_population.xlsx.
9. Format the header row, column widths and number format using openpyxl.
10. Display the first rows and a success message.
11. Stop the program.

CODE:

WIKI_TO_EXCEL.PY

```
import requests
import pandas as pd
from io import StringIO
from openpyxl import load_workbook
from openpyxl.styles import Font, PatternFill

URL =
"https://en.wikipedia.org/wiki/List_of_states_and_union_territories_of_India_by_population"
HEADERS = {"User-Agent": "Mozilla/5.0 (BPA Lab Scraper)"}

html = requests.get(URL, headers=HEADERS, timeout=15).text
tables = pd.read_html(StringIO(html))
print(f"Tables found on page: {len(tables)}")

df = tables[1].iloc[:, [1, 2]].copy()           # State name, Population (2011)
df.columns = ["State", "Population"]
df["Population"] = pd.to_numeric(df["Population"].astype(str)
                                 .str.replace(r"\[.*\]|,", "", regex=True),
errors="coerce")
df = df.dropna().sort_values("Population", ascending=False).head(15)
df.insert(0, "Rank", range(1, len(df) + 1))
df["Share (%)"] = (df["Population"] / 1210854977 * 100).round(2)

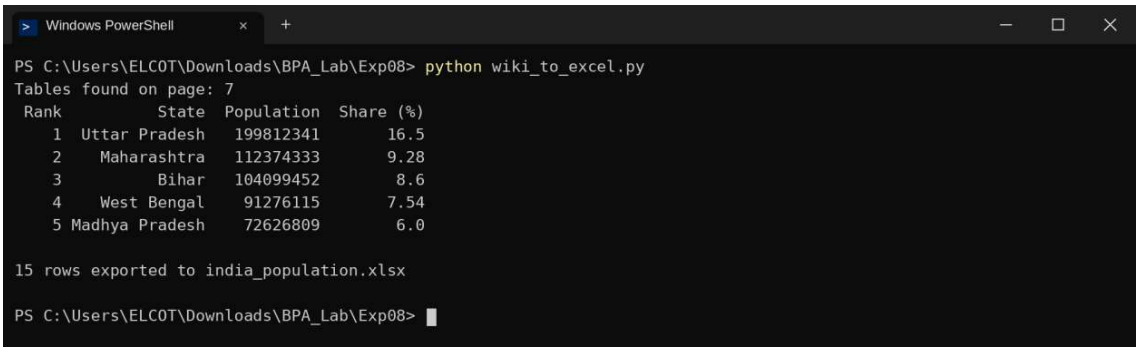
OUT = "india_population.xlsx"
df.to_excel(OUT, index=False, sheet_name="Top 15 States")

wb = load_workbook(OUT)
ws = wb.active
for cell in ws[1]:
    cell.font = Font(bold=True, color="FFFFFF")
```

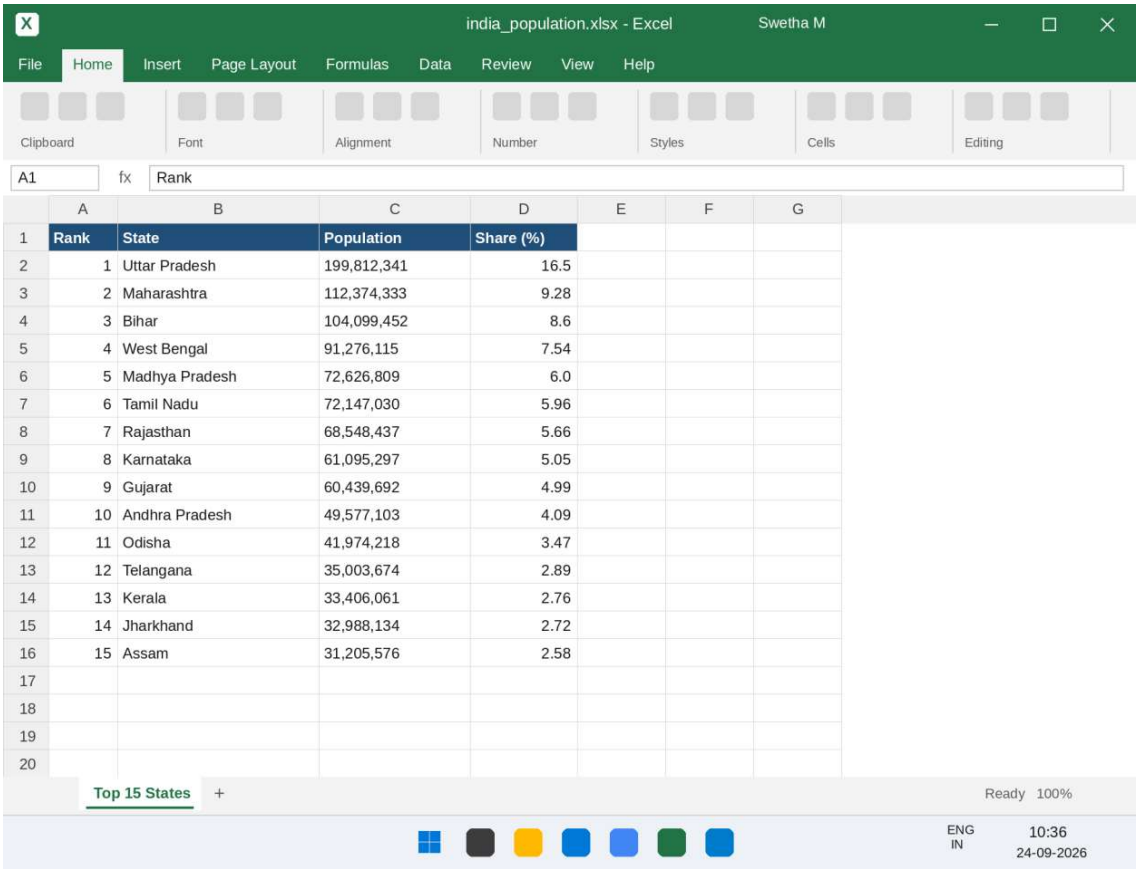
```
    cell.fill = PatternFill("solid", start_color="1F4E78")
for col, width in zip("ABCD", [7, 22, 16, 11]):
    ws.column_dimensions[col].width = width
for row in ws.iter_rows(min_row=2, min_col=3, max_col=3):
    row[0].number_format = "#,##0"
wb.save(OUT)

print(df.head().to_string(index=False))
print(f"\n{len(df)} rows exported to {OUT}")
```

OUTPUT:



Program execution in PowerShell



	A	B	C	D	E	F	G
1	Rank	State	Population	Share (%)			
2	1	Uttar Pradesh	199,812,341	16.5			
3	2	Maharashtra	112,374,333	9.28			
4	3	Bihar	104,099,452	8.6			
5	4	West Bengal	91,276,115	7.54			
6	5	Madhya Pradesh	72,626,809	6.0			
7	6	Tamil Nadu	72,147,030	5.96			
8	7	Rajasthan	68,548,437	5.66			
9	8	Karnataka	61,095,297	5.05			
10	9	Gujarat	60,439,692	4.99			
11	10	Andhra Pradesh	49,577,103	4.09			
12	11	Odisha	41,974,218	3.47			
13	12	Telangana	35,003,674	2.89			
14	13	Kerala	33,406,061	2.76			
15	14	Jharkhand	32,988,134	2.72			
16	15	Assam	31,205,576	2.58			
17							
18							
19							
20							

Formatted Excel output

RESULT:

Thus, the tabular data was successfully scraped from the web page, cleaned and exported into a formatted Excel file.

EXP-9: EXCEL-BASED AUTOMATION

DATE:

AIM:

To automate the processing of student marks stored in an Excel file by calculating total, average, grade and result, highlighting failures and generating a summary sheet with a chart using Python.

ALGORITHM:

1. Start the program.
2. Import load_workbook and the styling and chart classes from openpyxl.
3. Open marks_input.xlsx and select the active sheet.
4. Add the column headers Total, Average, Grade and Result.
5. For every student, read the subject marks and compute the total and average.
6. Assign the grade and decide PASS / FAIL (minimum 50 in every subject).
7. Highlight the rows of failed students in red.
8. Format the header row and adjust the column widths.
9. Create a Summary sheet with total, passed, failed and pass percentage.
10. Add a bar chart of average marks to the Summary sheet.
11. Save the workbook as marks_report.xlsx.
12. Stop the program.

CODE:

EXCEL_AUTOMATION.PY

```
"""
Exp 9: Excel-Based Automation
Reads student marks from marks_input.xlsx, computes Total, Average,
Grade and Result, highlights failures, adds a summary sheet with a chart
and writes marks_report.xlsx.
"""
from openpyxl import load_workbook
from openpyxl.styles import Font, PatternFill, Alignment
from openpyxl.chart import BarChart, Reference

IN_FILE = "marks_input.xlsx"
OUT_FILE = "marks_report.xlsx"


def grade(avg):
    if avg >= 90: return "O"
    if avg >= 80: return "A+"
    if avg >= 70: return "A"
    if avg >= 60: return "B+"
    if avg >= 50: return "B"
    return "RA"


def process():
    wb = load_workbook(IN_FILE)
    ws = wb.active
    subjects = ws.max_column - 2          # columns: RegNo, Name, subjects...
```

```
    last = ws.max_column
    for i, h in enumerate(["Total", "Average", "Grade", "Result"], start=1):
        ws.cell(row=1, column=last + i, value=h)

    red = PatternFill("solid", start_color="FFC7CE")
    passed = failed = 0
    for r in range(2, ws.max_row + 1):
        marks = [ws.cell(r, c).value for c in range(3, 3 + subjects)]
        total = sum(marks)
        avg = round(total / subjects, 2)
        result = "PASS" if min(marks) >= 50 else "FAIL"
        ws.cell(r, last + 1, total)
        ws.cell(r, last + 2, avg)
        ws.cell(r, last + 3, grade(avg) if result == "PASS" else "RA")
        ws.cell(r, last + 4, result)
        print(f"{ws.cell(r, 1).value}  {ws.cell(r, 2).value:<8} Total={total:<4}
Avg={avg:<6} Grade={ws.cell(r, last + 3).value:<3} {result}")
        if result == "FAIL":
            failed += 1
            for c in range(1, last + 5):
                ws.cell(r, c).fill = red
        else:
            passed += 1

    header_fill = PatternFill("solid", start_color="4F81BD")
    for c in ws[1]:
        c.font = Font(bold=True, color="FFFFFF")
        c.fill = header_fill
        c.alignment = Alignment(horizontal="center")
    for col in ws.columns:
        ws.column_dimensions[col[0].column_letter].width = max(len(str(x.value)) for
x in col) + 3

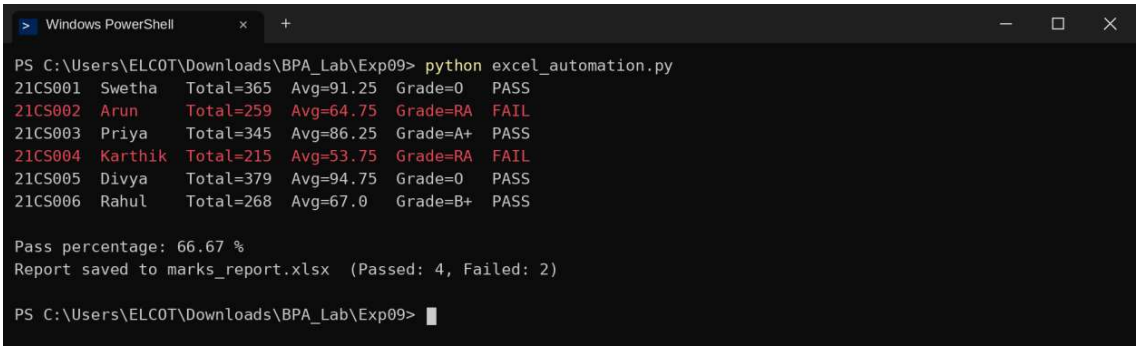
    # Summary sheet
    s = wb.create_sheet("Summary")
    s.append(["Metric", "Value"])
    s.append(["Total Students", passed + failed])
    s.append(["Passed", passed])
    s.append(["Failed", failed])
    s.append(["Pass %", round(passed / (passed + failed) * 100, 2)])
    s.column_dimensions["A"].width = 18

    chart = BarChart()
    chart.title = "Average Marks per Student"
    chart.add_data(Reference(ws, min_col=last + 2, min_row=1, max_row=ws.max_row),
titles_from_data=True)
    chart.set_categories(Reference(ws, min_col=2, min_row=2, max_row=ws.max_row))
    s.add_chart(chart, "D2")

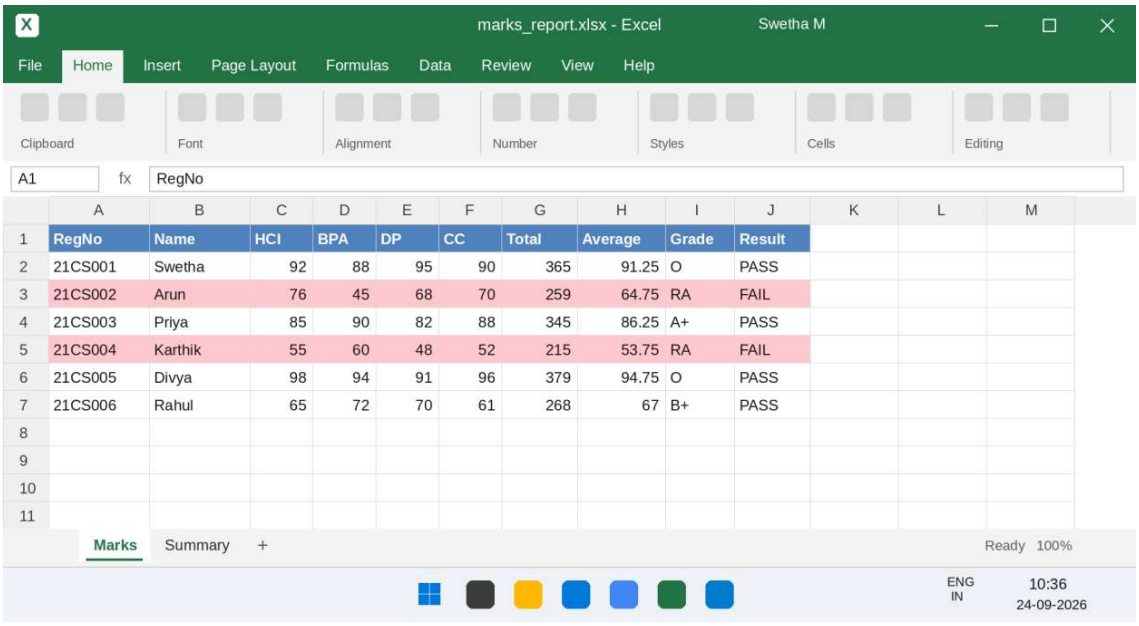
    wb.save(OUT_FILE)
    print(f"\nPass percentage: {round(passed / (passed + failed) * 100, 2)} %")
    print(f"Report saved to {OUT_FILE}  (Passed: {passed}, Failed: {failed})")


if __name__ == "__main__":
    process()
```

OUTPUT:



Program execution in PowerShell



Generated marks_report.xlsx with failures highlighted

RESULT:

Thus, the student marks in the Excel file were processed automatically. The total, average, grade and result were calculated, failures were highlighted and a summary sheet with a chart was generated.

EXP-10: PDF INVOICE EXTRACTION

DATE:

AIM:

To extract invoice details such as invoice number, date, vendor, customer, subtotal, GST and grand total from multiple PDF invoices and store them in an Excel file using Python.

ALGORITHM:

1. Start the program.
2. Import the os, re, pdfplumber and pandas modules.
3. Read the list of PDF files in the invoices folder.
4. Open each invoice using pdfplumber and extract the text.
5. Extract the invoice number, date, vendor and customer using regular expressions.
6. Extract the subtotal, GST and grand total and convert them to numbers.
7. Store the details of each invoice in a list.
8. Convert the list into a pandas DataFrame.
9. Export the DataFrame to invoice_summary.xlsx.
10. Display the number of invoices and the total amount.
11. Stop the program.

CODE:

INVOICE_READER.PY

```
import os
import re
import pdfplumber
import pandas as pd

FOLDER = "invoices"


def extract(pattern, text):
    match = re.search(pattern, text)
    return match.group(1).strip() if match else None

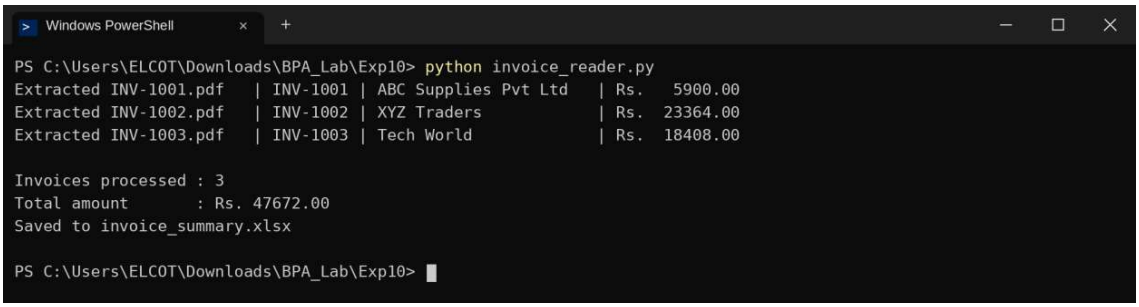

records = []
for file in sorted(os.listdir(FOLDER)):
    if not file.endswith(".pdf"):
        continue
    with pdfplumber.open(os.path.join(FOLDER, file)) as pdf:
        text = "\n".join(page.extract_text() for page in pdf.pages)

    record = {
        "Invoice No": extract(r"Invoice No\s*:\s*(\S+)", text),
        "Date": extract(r"Date\s*:\s*([\d-]+)", text),
        "Vendor": extract(r"Vendor\s*:\s*(.+)", text),
        "Bill To": extract(r"Bill To\s*:\s*(.+)", text),
        "Subtotal": float(extract(r"Subtotal\s*:\s*Rs\.\s*([\d.]+)", text)),
        "GST": float(extract(r"GST \(18%\)\s*:\s*Rs\.\s*([\d.]+)", text)),
        "Grand Total": float(extract(r"Grand Total\s*:\s*Rs\.\s*([\d.]+)", text)),
    }
    records.append(record)
```

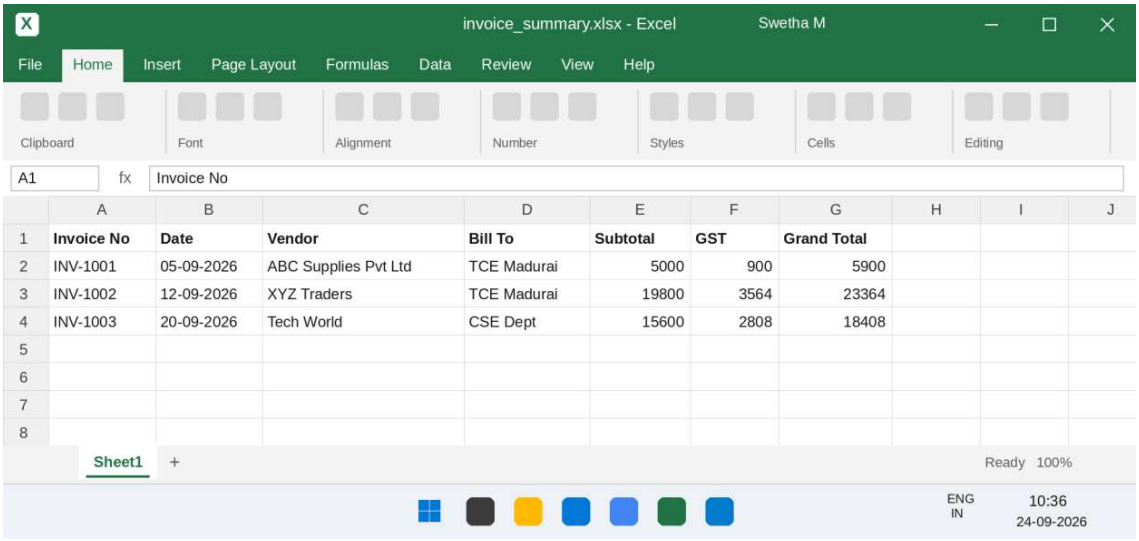
```
    print(f"Extracted {file:<14} | {record['Invoice No']} | {record['Vendor']:<22} |
Rs. {record['Grand Total']:>9.2f}")

df = pd.DataFrame(records)
df.to_excel("invoice_summary.xlsx", index=False)
print(f"\nInvoices processed : {len(df)}")
print(f"Total amount       : Rs. {df['Grand Total'].sum():.2f}")
print("Saved to invoice_summary.xlsx")
```

OUTPUT:



Program execution in PowerShell



Extracted invoice details in Excel

RESULT:

Thus, the required details were successfully extracted from all the PDF invoices and stored in an Excel file using Python.

EXP-11: EMAIL AUTOMATION AND ATTACHMENT DOWNLOAD

DATE:

AIM:

To automatically read unread emails with 'Invoice' in the subject from a Gmail inbox and download their attachments to a local folder using Python's imaplib module.

ALGORITHM:

1. Start the program.
2. Import the os, email, imaplib and decode_header modules.
3. Read the Gmail address and App Password from environment variables.
4. Create the attachments folder if it does not exist.
5. Connect to imap.gmail.com using IMAP4_SSL and log in.
6. Select the INBOX folder.
7. Search for unread mails whose subject contains 'Invoice'.
8. Fetch each mail and display the sender and subject.
9. Walk through the parts of each mail and save every attachment into the folder.
10. Log out and display the number of attachments saved.
11. Stop the program.

CODE:

ATTACHMENT_BOT.PY

```
import os
import email
import imaplib
from email.header import decode_header

USER = os.getenv("EMAIL_USER")
PASSWORD = os.getenv("EMAIL_PASS")          # Gmail App Password
SAVE_DIR = "attachments"
SEARCH = '(UNSEEN SUBJECT "Invoice")'


def decode(value):
    text, enc = decode_header(value)[0]
    return text.decode(enc or "utf-8") if isinstance(text, bytes) else text


os.makedirs(SAVE_DIR, exist_ok=True)
mail = imaplib.IMAP4_SSL("imap.gmail.com")
mail.login(USER, PASSWORD)
mail.select("INBOX")
print("Logged in to imap.gmail.com - INBOX selected")

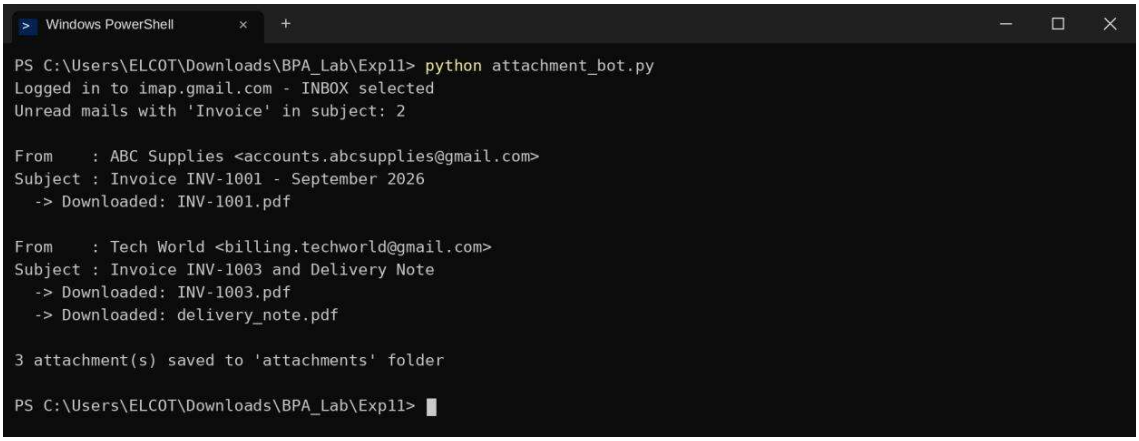
status, data = mail.search(None, SEARCH)
ids = data[0].split()
print(f"Unread mails with 'Invoice' in subject: {len(ids)}\n")

count = 0
for mid in ids:
    _, msg_data = mail.fetch(mid, "(RFC822)")
    msg = email.message_from_bytes(msg_data[0][1])
```

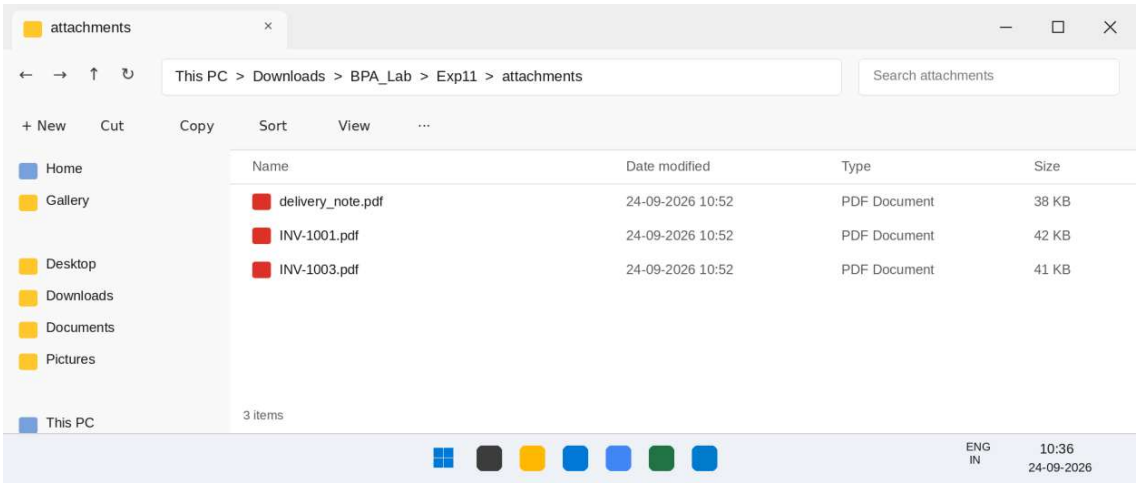
```
    print(f"From    : {decode(msg['From'])}")
    print(f"Subject : {decode(msg['Subject'])}")
    for part in msg.walk():
        if part.get_content_disposition() == "attachment":
            filename = decode(part.get_filename())
            with open(os.path.join(SAVE_DIR, filename), "wb") as f:
                f.write(part.get_payload(decode=True))
            count += 1
            print(f"  -> Downloaded: {filename}")
    print()

mail.logout()
print(f"{count} attachment(s) saved to '{SAVE_DIR}' folder")
```

OUTPUT:



Program execution in PowerShell



Attachments downloaded to the local folder

RESULT:

Thus, the unread invoice emails were read automatically from the Gmail inbox and their attachments were downloaded and saved to a local folder using Python.

EXP-12: EXCEPTION HANDLING AND LOGGING

DATE:

AIM:

To implement exception handling using try, except, else and finally with custom exceptions and retry logic, and to record program activities and errors using the logging module in Python.

ALGORITHM:

1. Start the program.
2. Import the logging, time and random modules.
3. Configure logging to write to both the console and automation.log.
4. Define the custom exceptions BusinessRuleException and SystemException.
5. Define a function to process a transaction and add 18% GST, raising exceptions for invalid data.
6. Define a retry function that retries SystemException up to three times.
7. For each transaction, call the process inside a try block.
8. Handle business, system and unexpected exceptions in separate except blocks.
9. Use else to log successful transactions and finally to log completion.
10. Demonstrate ZeroDivisionError handling.
11. Log the summary of the process.
12. Stop the program.

CODE:

EXCEPTION_LOGGING.PY

```
"""
Exp 12: Exception Handling and Logging
Processes a list of transactions with try/except/else/finally,
custom exceptions, retry logic, and logs to console + a log file.
"""
import logging
import time
import random

logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s | %(levelname)-8s | %(funcName)s | %(message)s",
    handlers=[logging.FileHandler("automation.log", mode="w"),
logging.StreamHandler()],
)
log = logging.getLogger("BPA")


class BusinessRuleException(Exception):
    """Invalid data - should NOT be retried (like UiPath BusinessRuleException)."""


class SystemException(Exception):
    """Temporary technical failure - can be retried."""
```

```
def process_transaction(txn):
    if txn["amount"] is None:
        raise BusinessRuleException(f"Amount missing for {txn['id']}")
    if not isinstance(txn["amount"], (int, float)):
        raise TypeError(f"Amount must be numeric, got
{type(txn['amount']).__name__}")
    if txn["amount"] <= 0:
        raise BusinessRuleException(f"Invalid amount {txn['amount']} for
{txn['id']}")
    if random.random() < 0.3:
        raise SystemException("Simulated network timeout")
    return round(txn["amount"] * 1.18, 2)  # add 18% GST


def with_retry(func, txn, retries=3, delay=0.5):
    for attempt in range(1, retries + 1):
        try:
            return func(txn)
        except SystemException as e:
            log.warning(f"{txn['id']} attempt {attempt}/{retries} failed: {e}")
            time.sleep(delay)
    raise SystemException(f"{txn['id']} failed after {retries} retries")


def main():
    random.seed(7)
    txns = [
        {"id": "T001", "amount": 1000},
        {"id": "T002", "amount": -50},
        {"id": "T003", "amount": None},
        {"id": "T004", "amount": "abc"},
        {"id": "T005", "amount": 2500},
        {"id": "T006", "amount": 750},
    ]
    stats = {"success": 0, "business": 0, "system": 0, "other": 0}
    log.info("=== Process started ===")

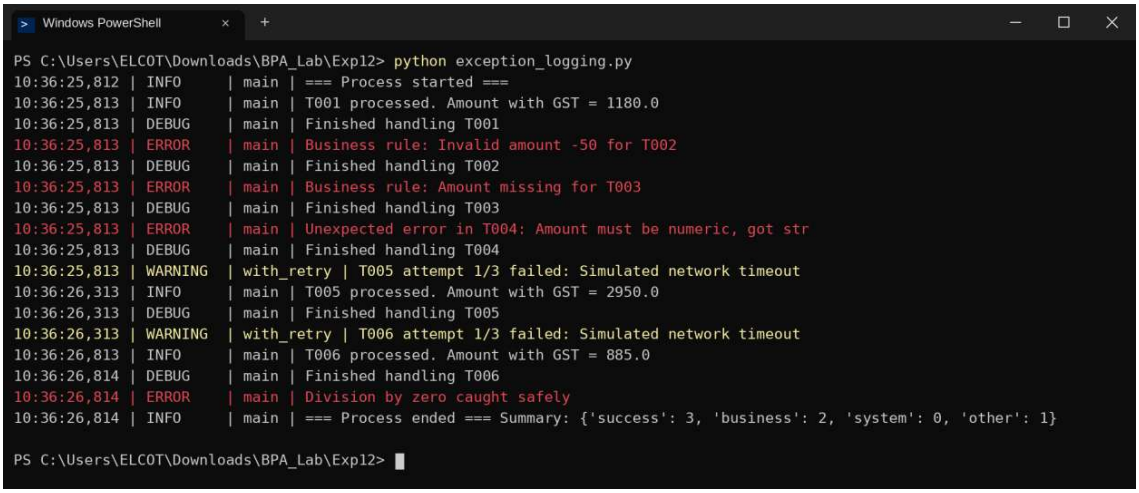
    for txn in txns:
        try:
            result = with_retry(process_transaction, txn)
        except BusinessRuleException as e:
            stats["business"] += 1
            log.error(f"Business rule: {e}")
        except SystemException as e:
            stats["system"] += 1
            log.critical(f"System: {e}")
        except Exception as e:
            stats["other"] += 1
            log.error(f"Unexpected error in {txn['id']}: {e}")
        else:
            stats["success"] += 1
            log.info(f"{txn['id']} processed. Amount with GST = {result}")
        finally:
            log.debug(f"Finished handling {txn['id']}")

    # Division by zero demo
    try:
        rate = stats["success"] / 0
    except ZeroDivisionError:
        log.error("Division by zero caught safely")

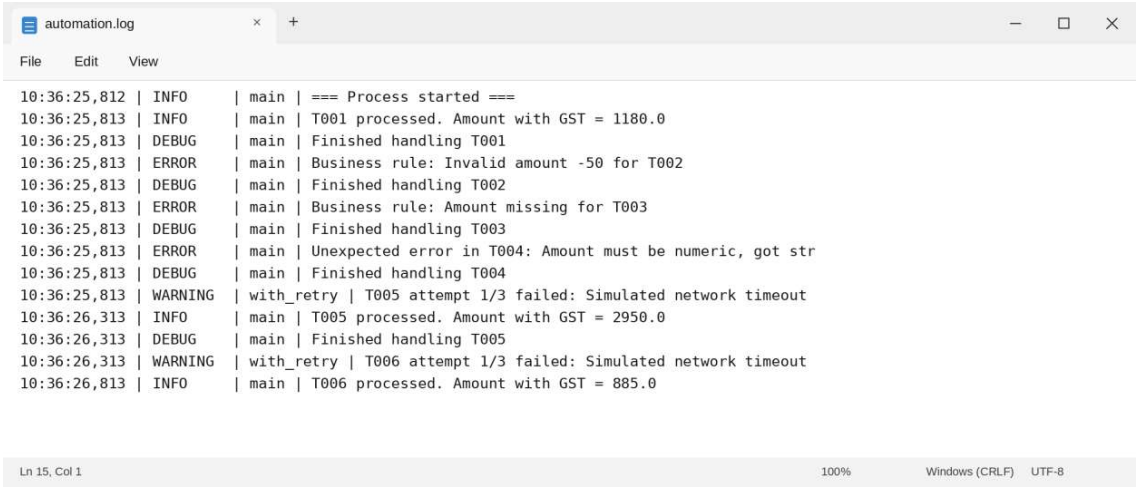
    log.info(f"=== Process ended === Summary: {stats}")


if __name__ == "__main__":
    main()
```

OUTPUT:



Program execution in PowerShell



Contents of automation.log

RESULT:

Thus, exception handling was successfully implemented using try, except, else and finally along with custom exceptions and retry logic, and all program events and errors were recorded using logging.